

Sujet de TP : Observabilité Avancée et Architecture Distribuée (FastAPI, Prometheus, ELK)

Environnement : Deux machines virtuelles Linux (Debian 13 / Ubuntu) interconnectées par un réseau privé.

Architecture Réseau Cible

Pour ce TP, l'utilisation de Docker est **strictement interdite sur la VM 1**. Tous les services doivent y être installés et configurés de manière traditionnelle (Bare-Metal).

- **VM 1 (Cible/Production)** : IP `<IP_VM_CIBLE>`. Héberge l'application FastAPI et l'agent système Node Exporter en tant que services `systemd`.
- **VM 2 (Monitoring/Supervision)** : IP `<IP_VM_MONITORING>`. Héberge les serveurs Prometheus, Grafana, Elasticsearch, Logstash et Kibana.

Configuration "Bare-Metal" et Sécurisation de la VM 1

Objectifs

- Installer et configurer des démons Linux via `systemd`.
- Sécuriser les accès réseau aux métriques via un pare-feu local.

Travail à réaliser

1. Installation et persistance de Node Exporter

Prometheus a besoin d'un agent pour lire les compteurs de performance de Linux (CPU, RAM, Disques).

1. Téléchargez l'archive officielle Linux `tar.gz` depuis le site de Prometheus sur la **VM 1**.
2. Extrayez les binaires et déplacez le fichier exécutable dans `/usr/local/bin/`.
3. **Sécurité** : Créez un utilisateur système dédié, sans privilèges et sans shell, nommé `node_exporter` :
4. **Persistance** : Créez un fichier de service `systemd` nommé `/etc/systemd/system/node_exporter.service` afin que l'agent démarre automatiquement avec la machine.

5. Activez et lancez le service (`systemctl enable --now node_exporter`). Vérifiez son bon fonctionnement avec `systemctl status`.

2. Sécurisation des flux (Pare-feu UFW)

Par défaut, vos métriques système ne doivent pas être exposées à n'importe qui.

1. Activez le pare-feu `ufw` sur la **VM 1**.
2. Configurez une règle stricte pour autoriser **uniquement la VM 2** à interroger le port `9100`. Bloquez tous les autres accès.

Développement et Instrumentation de l'API FastAPI

Objectifs

- Déployer une application asynchrone moderne sous forme de service d'infrastructure.
- Implémenter un Middleware pour intercepter et mesurer globalement le trafic HTTP.

Enoncé et Travail à réaliser

1. Code de base fourni

Créez un dossier `/home/etudiant/app_fastapi` sur la **VM 1**. Vous devez mettre en place un environnement virtuel Python (`python3 -m venv venv`) et y installer `fastapi`, `uvicorn` et `prometheus-client`.

2. Instrumentation Prometheus via Middleware

Plutôt que d'instrumenter chaque route individuellement, vous devez utiliser le mécanisme de **Middleware** de FastAPI pour capturer toutes les requêtes de manière transverse.

1. Modifiez `app.py` pour déclarer deux métriques Prometheus :
 - Un **Counter** nommé `api_requests_total` avec les labels `method`, `endpoint`, et `status_code`.
 - Un **Histogram** nommé `api_request_duration_seconds` avec les labels `method` et `endpoint`.
2. Exposez le endpoint réglementaire `/metrics` en utilisant l'adaptateur ASGI officiel (`make_asgi_app`).
3. Implémentez le middleware `@app.middleware("http")` pour calculer le temps d'exécution réel de chaque requête et incrémenter vos compteurs.
4. Créez un fichier de service `/etc/systemd/system/api-fastapi.service` pour gérer l'exécution en arrière-plan via `uvicorn` sur le port `5000` (Assurez-vous d'écouter sur `0.0.0.0`).

- Adaptez le pare-feu UFW de la **VM 1** pour que seule la **VM 2** puisse scraper le port **5000**.

Collecte Prometheus et Visualisation Grafana (sur VM 2)

Objectifs

- Centraliser les métriques distantes.
- Maîtriser le langage PromQL et concevoir un tableau de bord professionnel.

Enoncé et Travail à réaliser

1. Configuration de Prometheus

Sur la **VM 2**, installez et lancez Prometheus. Modifiez le fichier `prometheus.yml` pour cibler la **VM 1**

2. Exercices PromQL obligatoires (à consigner dans le rapport)

Trouvez les requêtes PromQL permettant d'extraire :

- Le taux moyen de requêtes par seconde sur l'API lors des 5 dernières minutes.
- Le pourcentage de requêtes en erreur HTTP 500 par rapport au volume total.
- L'utilisation CPU de la VM 1 calculée à partir des données de `node_exporter`.

3. Dashboard Grafana

Déployez Grafana sur la **VM 2**, connectez votre source de données Prometheus et créez un tableau de bord comprenant :

- Un indicateur d'état (UP/DOWN) de la VM 1 et de l'API.
- Un graphique temporel empilé affichant le trafic par code de retour (200 vs 500).
- Un graphique de distribution des latences basé sur votre histogramme.

Migration vers la Stack ELK pour la Centralisation des Logs

Les métriques indiquent qu'il y a des erreurs, les logs expliquent pourquoi. Vous allez migrer l'analyse événementielle vers une stack **ELK** (Elasticsearch, Logstash, Kibana) hébergée sur la **VM 2**.

Objectifs

- Expédier les fichiers de logs système d'une machine à une autre.
- Parser une chaîne de caractères brute (logs Uvicorn) en données structurées.

Enoncé et Travail à réaliser

1. Expédition des logs depuis la VM 1

1. Installez l'agent léger **Filebeat** directement sur la **VM 1**.
2. Configurez le fichier `/etc/filebeat/filebeat.yml` pour lire les logs de votre service système FastAPI. (Indice : *Uvicorn envoie ses logs dans syslog ou dans les fichiers capturés par journald, assurez-vous d'orienter Filebeat vers le bon flux ou configurez l'écriture d'un fichier dédié `/var/log/api-fastapi.log`*).
3. Configurez la sortie (`output.logstash`) pour envoyer les données vers `<IP_VM_MONITORING>:5044`.

2. Réception et Parsing Grok sur la VM 2

Les logs générés nativement par Uvicorn ressemblent à ceci :

INFO: 192.168.56.10:45322 - "GET /traitement HTTP/1.1" 200 OK

Sur la **VM 2**, configurez le pipeline de traitement dans votre fichier `logstash.conf` :

1. Déclarez l'entrée sur le port d'écoute `5044`.
2. **Le défi Grok** : Écrivez un filtre `grok` capable de découper cette ligne brute pour isoler dynamiquement les champs suivants : `loglevel`, `client_ip`, `method`, `request`, et `response_code`.
3. Configurez la sortie pour pousser ces données documentées dans un index Elasticsearch nommé `logs-fastapi-%{+YYYY.MM.dd}`.

3. Analyse sous Kibana

1. Accédez à Kibana (`http://<IP_VM_MONITORING>:5601`).
2. Créez une **Data View** indexée sur vos logs.
3. Construisez un tableau de bord Kibana présentant :
 - Un camembert (Pie Chart) de répartition des codes d'erreur (champs `response_code`).
 - Un top 5 des adresses IP qui sollicitent le plus l'API (`client_ip`).

Livrables attendus pour l'évaluation

Chaque étudiant devra fournir une archive contenant :

1. **Architecture & Système** : Les fichiers de service `systemd` créés sur la VM 1 et les règles de pare-feu appliquées.
2. **Code** : Le script `app.py` final contenant l'application FastAPI, son middleware et ses métriques.
3. **Pipelines** : Le fichier de configuration `logstash.conf` contenant le filtre Grok fonctionnel.

4. **Rapport (PDF)** : * Les réponses détaillées aux questions PromQL.
- Une capture d'écran du menu *Discover* de Kibana prouvant qu'un log Uvicorn a été correctement découpé en champs exploitables.
 - Une capture d'écran de l'interface Prometheus montrant une alerte active lorsque le service `api-fastapi` est arrêté manuellement (`systemctl stop`).

Rendre le rapport "Analytique" plutôt que "Technique"

Au lieu de fournir des captures d'écran du code , analysez ce que vous constatez :

- **Analyse de performance** : *"Faites un tir de charge sur votre API FastAPI (avec un outil comme `ab` ou `wrk`). Observez l'évolution du graphique de latence sur Grafana. À quel percentile commencez-vous à voir une dégradation des performances ? Pourquoi ?"*
- **Explication de texte** : Documenter, dans le rapport, chaque ligne du filtre Grok fourni par l'IA.

Introduire une phase obligatoire de "Chaos Engineering" (Pannes complexes)

L'IA est très forte pour construire, elle est plus limitée pour diagnostiquer une panne en temps réel sur une infra spécifique. Défis en fin de TP :

- **La panne réseau** : *"Bloquez un port spécifique sur le pare-feu de la VM 1. Que se passe-t-il sur Prometheus ? Que devient la valeur de la métrique `up` ? Combien de temps met l'alerte pour passer de Pending à Firing ? Observez-vous un impact sur Kibana ?"*
- **La panne de disque / saturation** : *"Générez une boucle infinie qui sature la RAM de la VM 1. À partir de quel seuil l'alerte système se déclenche-t-elle ?"*

Le juge de paix

Réfléchissez aux questions suivantes :

- *"Montrez-moi votre middleware FastAPI. Si je lance une requête qui met 3 secondes à répondre, quelle ligne de votre code calcule cette durée et où est-elle stockée dans Prometheus ?"*
- *"Quelle est la différence fondamentale entre la manière dont Prometheus récupère les données (Pull) et la manière dont Filebeat/Logstash les envoient (Push) ? Pourquoi avoir choisi ces deux modèles différents pour les métriques et les logs ?"*
- *"Si Elasticsearch crash, que deviennent les logs envoyés par Filebeat pendant la panne ? Sont-ils perdus ?"*